

Mô hình camera và méo ảnh (Camera Model and Distortion)

PHỤ THUỘC Không có — đây là chương mở đầu của cây. Nó chỉ giả định bạn biết tọa độ thuần nhất và phép nhân ma trận. Mọi chương còn lại của Phần A đều gọi về đây, và A/05 lấy từ đây đại lượng f mà mọi công thức sai số của nó đều chia cho.

TÓM TẮT

Chương này dựng mô hình biến một điểm trong không gian thành một pixel, và nó tồn tại vì hai lý do rất cụ thể chứ không phải vì tính đầy đủ. Lý do thứ nhất: nó cấp *tiêu cự tính bằng pixel* f , đại lượng camera duy nhất xuất hiện trong mọi công thức sai số của cây. Lý do thứ hai, và đây mới là lý do chính: nó giải thích hai cơ chế hỏng mà người làm marker gặp thường xuyên và hầu như không bao giờ chẩn đoán đúng. Cơ chế một — méo ảnh chưa khử làm *cong* cạnh tag, mà detector thì khớp *đường thẳng* lên cạnh, nên giao điểm hai đường thẳng sai, nên góc sai, nên pose sai. Cơ chế hai — điểm chính (principal point) sai gây ra một thiên lệch xoay *có hệ thống* phụ thuộc vị trí tag trên ảnh. Nếu chỉ nhớ một điều: *méo ảnh không gây nhiễu, nó gây thiên lệch* — và thiên lệch thì không trung bình mất đi dù lấy bao nhiêu khung.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tọa độ thuần nhất (homogeneous coordinates)** — cách viết một điểm 2D bằng ba số (x, y, w) với quy ước $(x, y, w) \sim (\lambda x, \lambda y, \lambda w)$ cho mọi $\lambda \neq 0$; điểm Euclid tương ứng là $(x/w, y/w)$. Nó tồn tại để phép chiếu phối cảnh — vốn là một phép chia — viết được thành một phép nhân ma trận.
- **tọa độ chuẩn hoá (normalized coordinates)** — tọa độ của điểm sau khi chia cho chiều sâu nhưng *trước* khi nhân với K : $(x_n, y_n) = (X/Z, Y/Z)$. Đây là hệ tọa độ mà mọi mô hình méo được định nghĩa trên đó, và nhầm lẫn giữa nó với tọa độ pixel là lỗi cài đặt phổ biến nhất khi tự viết code undistort.
- **tiêu cự tính bằng pixel** — $f_x = F/s_x$ với F là tiêu cự vật lý của ống kính và s_x là kích thước một pixel. Nó *không* phải một chiều dài; nó là một tỉ số không thứ nguyên, và đó là lý do nó xuất hiện được ở mẫu số của các công thức sai số.
- **điểm chính (principal point)** — giao điểm của trục quang với mặt phẳng ảnh, ký hiệu (c_x, c_y) . Trục giác nói nó nằm ở giữa ảnh; thực tế nó lệch vài pixel tới vài chục pixel do sai lệch lắp ráp, và mục 5 giải thích vì sao sai lệch ấy nguy hiểm hơn vẻ ngoài.
- **méo xuyên tâm (radial distortion)** — sai lệch mà độ lớn chỉ phụ thuộc khoảng cách từ điểm chính, theo hướng ra xa hoặc vào gần tâm. Đây là thành phần méo lớn nhất trong hầu hết ống kính.
- **méo tiếp tuyến (tangential distortion)** — sai lệch theo hướng vuông góc với bán kính, sinh ra khi mặt phẳng cảm biến không song song hoàn hảo với mặt phẳng thấu kính. Thường nhỏ hơn méo xuyên tâm một bậc, nhưng nó là thành phần *bất đối xứng* nên nó không tự triệt tiêu.
- **sai số mô hình và sai số tham số** — sai số tham số là khi bạn dùng đúng dạng hàm nhưng ước lượng sai hệ số; sai số mô hình là khi dạng hàm bản thân nó không đủ để mô tả hiện thực. Loại thứ hai nguy hiểm hơn vì thêm dữ liệu không chứa được nó, và mục 6 dành riêng cho việc phân biệt hai loại.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn hiệu chuẩn camera, sai số tái chiếu trung bình là 0,25 px — một con số nghe rất tốt. Vì sao con số ấy có thể đang che giấu một lỗi mô hình nghiêm trọng, và bạn nhìn vào đâu để phát hiện?
- Q2. Detector AprilTag khớp một đường *thẳng* lên mỗi cạnh tag rồi lấy giao điểm làm góc. Nếu ảnh còn méo 3 px ở rìa, sai số góc mà bạn nhận được là bao nhiêu — và vì sao câu trả lời không phải “3 px”?
- Q3. Điểm chính của bạn bị sai 10 px so với giá trị thật. Sai số ấy gây ra hậu quả gì cho góc của pose, và vì sao hậu quả ấy thay đổi khi tag di chuyển trong khung hình?
- Q4. Có hai cách xử lý méo: khử méo toàn ảnh rồi chạy detector, hoặc chạy detector trên ảnh gốc rồi khử méo cho từng toạ độ góc. Cách nào chính xác hơn, và cách nào nhanh hơn — và vì sao câu trả lời cho hai câu ấy không giống nhau?
- Q5. Bạn dùng mô hình hai hệ số k_1, k_2 cho một ống kính góc rộng 120° . Hiệu chuẩn hội tụ, residual nhỏ. Vì sao điều đó *không* chứng minh mô hình của bạn đủ, và dấu hiệu nào sẽ lộ ra sự thiếu?

1.1 Đặt vấn đề

Mọi thứ trong cây này đều bắt đầu từ một phép biến đổi: một điểm ở đâu đó trong không gian trở thành một toạ độ pixel trên một tấm ảnh. Nếu phép biến đổi ấy được mô tả đúng, mọi công thức phía sau có nghĩa. Nếu nó được mô tả sai — kể cả sai một chút — thì mọi thứ phía sau là chính xác quanh một giá trị sai.

Bài toán gốc thuộc về ngành trắc lượng ảnh (photogrammetry) chứ không phải thị giác máy tính, và nó đã có từ trước khi có máy tính: người ta cần đo đặc địa hình từ ảnh chụp trên không, và để làm được thì phải biết chính xác ống kính đã biến dạng ảnh như thế nào. Mô hình méo vẫn đang dùng ngày nay mang tên Brown–Conrady và được công bố ở dạng hiện đại năm 1971 [1]; phương pháp hiệu chuẩn từ một target phẳng mà mọi hộp công cụ đang dùng thì đến sau đó gần ba mươi năm [2].

Điều làm chương này đáng đọc với người làm marker — thay vì chỉ tra một công thức khi cần — nằm ở một quan sát mà tôi cho là chưa được nói đủ rõ ở đâu cả. Méo ảnh ảnh hưởng tới các bài toán thị giác theo những cách rất khác nhau, và nó ảnh hưởng tới *phát hiện marker* theo một cách đặc biệt xấu. Lý do là thế này: hầu hết thuật toán thị giác dùng đặc trưng điểm, và một điểm bị méo vẫn là một điểm — nó chỉ nằm sai chỗ, và khử méo là đủ. Nhưng detector marker không làm việc với điểm; nó làm việc với *cạnh thẳng*. AprilTag khớp một đường thẳng lên mỗi cạnh của tứ giác rồi lấy giao của bốn đường thẳng ấy làm bốn góc [3]. Và phép méo xuyên tâm biến một đường thẳng thành một *đường cong*. Khớp một đường thẳng lên một đường cong thì được một đường thẳng sai, và giao của hai đường thẳng sai thì sai nhiều hơn cả hai — mục 4 định lượng chuyện này.

Cách hiển nhiên để xử lý là khử méo toàn ảnh trước khi chạy detector, và cách ấy đúng. Chỗ nó không đủ nằm ở hai điểm mà mục 6 và mục 7 bàn tới: khử méo chỉ tốt đúng bằng mô hình méo bạn đã ước lượng, và nếu mô hình *sai dạng* — chẳng hạn dùng hai hệ số cho một ống kính góc rộng — thì phần méo còn sót lại không phải nhiễu ngẫu nhiên mà là một mẫu hình có cấu trúc, lớn nhất ở đúng chỗ tệ nhất là rìa ảnh.

Còn một lý do thứ hai để chương này không phải một phụ lục, và nó là lý do làm tôi xếp chương này ở mức xương sống thay vì mức tham khảo. Trong tất cả các tham số nội của camera, có một tham số mà sai lệch của nó gây hậu quả không tương xứng với vẻ ngoài của nó: điểm chính. Sai điểm chính không làm ảnh trông khác đi chút nào — không ai nhìn ảnh mà phát hiện được — nhưng nó tạo ra một thiên lệch trong *hướng* của pose, và thiên lệch ấy thay đổi theo vị trí tag trong khung hình nên nó trông hết như một sai số phụ thuộc tư thế. Mục 5 dành riêng cho nó.

TRỰC GIÁC

Hình dung camera như một cái thấu.

Đỉnh thấu là tâm quang học — một điểm. Mọi tia sáng đi tới ảnh đều phải chui qua đúng điểm ấy. Đáy thấu là mặt phẳng ảnh, đặt cách đỉnh một khoảng f . Một điểm trong không gian được chiếu lên ảnh bằng cách kẻ một đường thẳng từ nó tới đỉnh thấu và xem đường ấy cắt mặt phẳng ảnh ở đâu.

Mô hình pinhole nói rằng thấu là hoàn hảo: mọi tia đi thẳng, đỉnh thấu là một điểm toán học, mặt phẳng ảnh vuông góc với trục. Ống kính thật không phải thế. Thấu kính bẻ tia sáng, và nó bẻ các tia ở rìa nhiều hơn các tia ở giữa — đó là méo xuyên tâm. Cảm biến không lắp vuông góc tuyệt đối — đó là méo tiếp tuyến. Trục quang không đi qua đúng giữa cảm biến — đó là điểm chính lệch.

Ba sai lệch ấy khác nhau về bản chất theo một cách quan trọng. Hai cái đầu làm ảnh *trông* khác: đường thẳng cong đi, và mắt người phát hiện được nếu chụp một cái cửa sổ. Cái thứ ba thì không: dịch cả ảnh sang trái ba pixel thì không ai nhìn ra, vì không có gì trong ảnh để so sánh. Đó là lý do cái thứ ba nguy hiểm nhất — nó là sai lệch duy nhất không có triệu chứng thị giác.

1.2 Mô hình pinhole và ma trận K

Dựng mô hình theo ba bước, mỗi bước một phép biến đổi, vì việc tách bạch ba bước ấy là điều kiện để biết mình đang làm việc trong hệ tọa độ nào.

Bước một, từ thế giới sang camera. Một điểm $\mathbf{X}_w$ trong hệ thế giới được đưa sang hệ camera bằng một phép biến đổi cứng:

$$\mathbf{X}_c = R\mathbf{X}_w + \mathbf{t},$$

với $R \in SO(3)$ và $\mathbf{t} \in \mathbb{R}^3$. Cặp $(R, \mathbf{t})$ chính là *pose* mà toàn bộ cây này tìm cách ước lượng; sáu tham số của nó là sáu ẩn của bài toán PnP ở A/03.

Bước hai, chiếu phối cảnh. Chia cho chiều sâu:

$$\begin{pmatrix} x_n \\ y_n \end{pmatrix} = \begin{pmatrix} X_c/Z_c \\ Y_c/Z_c \end{pmatrix}.$$

Đây là tọa độ chuẩn hoá. Chúng không thứ nguyên, chúng là tang của góc nhìn theo hai hướng, và — quan trọng nhất cho mục đích của ta — *mọi mô hình méo được định nghĩa trên hệ tọa độ này*, không phải trên pixel. Đây là bước duy nhất trong ba bước mang tính phi tuyến, và toàn bộ độ khó của bài toán pose nằm ở phép chia này.

Bước ba, từ tọa độ chuẩn hoá sang pixel. Nhân với ma trận nội:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x_d \\ y_d \\ 1 \end{pmatrix}, \quad K = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

với (x_d, y_d) là tọa độ chuẩn hoá *đã méo* (mục 3). Bốn tham số thật sự quan trọng là f_x, f_y, c_x, c_y ; hệ số trượt s mô tả trường hợp trục pixel không vuông góc và với cảm biến số hiện đại thì nó bằng không tới mức không đo được — đừng ước lượng nó trừ khi có lý do cụ thể, vì một tham số dư luôn hút một phần sai số của các tham số khác.

Hai điều đáng dừng lại về f_x và f_y . Thứ nhất, chúng là *tỉ số không thứ nguyên*, không phải chiều dài — $f_x = F/s_x$ với F là tiêu cự vật lý và s_x là bề rộng một pixel. Đây là lý do f đứng được ở

mẫu số trong $\delta x \approx Z\sigma/f$ mà vẫn ra đúng thứ nguyên chiều dài. Thứ hai, có một cách rất nhanh để ước lượng f mà không cần hiệu chuẩn, dùng khi thiết kế: với trường nhìn ngang θ_{FOV} và độ phân giải ngang N_x pixel,

$$f_x \approx \frac{N_x/2}{\tan(\theta_{\text{FOV}}/2)}.$$

Ví dụ: camera 1280 px ngang, FOV 60°. Ta có $f_x \approx 640/\tan 30^\circ = 640/0,577 = 1109$ px. Con số ấy đủ chính xác để lập ngân sách sai số ở C/15 trước khi có camera trong tay — và đó chính là cách dùng nó.

1.3 Các mô hình méo

Méo được mô hình hoá như một phép biến đổi phi tuyến *trên toạ độ chuẩn hoá*, chen vào giữa bước hai và bước ba: $(x_n, y_n) \mapsto (x_d, y_d)$.

A. Brown–Conrady: radial cộng tangential

Mô hình phổ biến nhất, và là mặc định của OpenCV. Đặt $r^2 = x_n^2 + y_n^2$:

$$\begin{aligned} x_d &= x_n \underbrace{(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)}_{\text{xuên tâm}} + \underbrace{2p_1 x_n y_n + p_2 (r^2 + 2x_n^2)}_{\text{tiếp tuyến}}, \\ y_d &= y_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y_n^2) + 2p_2 x_n y_n. \end{aligned} \quad (1.1)$$

Đây là dạng chuẩn [1], viết theo quy ước của [2].

Ba điều cần đọc ra. Thứ nhất, thành phần xuyên tâm chỉ chứa *luỹ thừa chẵn* của r — đó không phải tùy tiện mà là hệ quả của tính đối xứng quay của thấu kính: nếu phép méo đối xứng quanh trục thì nó phải là hàm chẵn. Thứ hai, thành phần xuyên tâm *giữ nguyên hướng* của điểm so với tâm và chỉ đổi bán kính; hệ quả là một đường thẳng đi *qua* điểm chính vẫn thẳng sau khi méo, còn mọi đường thẳng khác thì cong. Ghi nhớ điều này — nó là chìa khoá của mục 4. Thứ ba, $k_1 < 0$ cho méo thùng (barrel, điển hình của ống kính góc rộng: các điểm ở rìa bị kéo vào trong) và $k_1 > 0$ cho méo gối (pincushion).

B. Mô hình hữu tỉ (rational)

Với ống kính góc rất rộng, đa thức bậc sáu không đủ. Mô hình hữu tỉ thay đa thức bằng một phân thức:

$$x_d = x_n \cdot \frac{1 + k_1 r^2 + k_2 r^4 + k_3 r^6}{1 + k_4 r^2 + k_5 r^4 + k_6 r^6} + (\text{tangential}).$$

Nó linh hoạt hơn nhiều nhưng nó có nhiều tham số hơn, và nhiều tham số hơn nghĩa là dễ khớp quá mức (overfitting) nếu dữ liệu hiệu chuẩn không phủ kín trường nhìn — một cạm bẫy mà B/12 bàn kỹ.

C. Mô hình fisheye Kannala–Brandt (KB) và các mô hình góc

Với ống kính trên khoảng 120°, toàn bộ họ mô hình trên gặp một vấn đề cấu trúc chứ không phải vấn đề độ chính xác: phép chiếu pinhole $r_n = \tan \theta$ phân kỳ khi $\theta \rightarrow 90^\circ$, nên không đa thức nào sửa được. Mô hình KB bỏ hẳn pinhole và mô hình hoá trực tiếp quan hệ giữa *góc tới* θ và bán kính ảnh [4]:

$$r_d = \theta + k_1 \theta^3 + k_2 \theta^5 + k_3 \theta^7 + k_4 \theta^9.$$

Đây là mô hình mặc định cho fisheye trong hầu hết hộp công cụ hiện đại, và có các mô hình mới hơn với ít tham số hơn cho cùng độ chính xác, chẳng hạn Double Sphere [5].

Bảng 1.1. Bốn họ mô hình méo, phạm vi áp dụng, và chế độ hỏng.

Mô hình	Số tham số	Dùng cho	Hỏng khi
Radial 2 hệ số (k_1, k_2)	2	Ống kính hẹp, dưới $\sim 50^\circ$	FOV rộng: residual có cấu trúc ở rìa, lớn dần theo r
Brown–Conrady đầy đủ	5	Phổ thông, $50\text{--}90^\circ$	Fisheye: dạng hàm không đủ dù thêm bao nhiêu ảnh
Hữu tỉ (rational)	8	Góc rộng $90\text{--}120^\circ$	Dữ liệu hiệu chuẩn không phủ rìa: khớp quá mức, ngoại suy sai
KB / fisheye	4	Trên 120° , tới $180^\circ+$	Dùng nhằm cho ống kính hẹp: tham số hoá kém điều kiện

1.4 Vì sao méo phá hoại detector marker một cách đặc biệt

Đây là mục quan trọng nhất của chương, và nó là chỗ chương này khác một chương về mô hình camera trong sách giáo khoa.

A. Cơ chế: khớp thẳng lên đường cong

AprilTag định vị góc theo một quy trình rất cụ thể: sau khi tìm được tứ giác ứng viên, nó khớp một *đường thẳng* lên mỗi cạnh, dùng trọng số theo độ lớn gradient, rồi lấy giao của các cặp đường thẳng kề nhau làm bốn góc [3]. Cách làm ấy chính xác hơn nhiều so với việc lấy trực tiếp điểm cực trị của contour, vì nó dùng *toàn bộ* chiều dài cạnh — hàng chục tới hàng trăm pixel — để xác định một đường, thay vì dựa vào vài pixel quanh góc.

Nhưng nó có một giả định ngầm: *cạnh của tag là một đường thẳng trên ảnh*. Giả định ấy đúng với mô hình pinhole (ảnh phối cảnh của một đường thẳng là một đường thẳng) và *sai* khi ảnh còn méo. Như mục 3A đã chỉ ra, méo xuyên tâm giữ nguyên hướng của điểm so với tâm nhưng đổi bán kính theo một hàm phi tuyến, nên một đường thẳng không đi qua điểm chính trở thành một đường cong.

Hậu quả có ba tầng, và cả ba đều đi theo chiều xấu.

Tầng một, đường thẳng khớp bị lệch. Khớp bình phương tối thiểu một đường thẳng lên một cung cong cho ra một đường thẳng đi qua giữa cung — nó cắt cung ở hai chỗ và lệch khỏi cung ở hai đầu và ở giữa, theo hướng ngược nhau.

Tầng hai, và đây là tầng quyết định: sai số được khuếch đại tại giao điểm. Hai đường thẳng gần vuông góc giao nhau thì sai số góc nhỏ của mỗi đường chuyển thành sai số vị trí ở giao điểm theo một hệ số phụ thuộc góc giao. Với hai cạnh của một tag nhìn chếch, góc giao có thể khá nhỏ, và khi đó hệ số khuếch đại lớn. Đây là câu trả lời cho Q2: sai lệch của góc *không* bằng sai lệch của cạnh — nó có thể lớn hơn nhiều.

Tầng ba, sai số là thiên lệch chứ không phải nhiễu. Với một tag ở một vị trí cố định trên ảnh, méo luôn đẩy cạnh theo cùng một hướng, mỗi khung hình như nhau. Lấy trung bình một nghìn khung không giúp gì — đây chính là loại nguồn “hệ thống” ở A/05 mục 8, loại không chia cho $\sqrt{N}$.

B. Ví dụ số: méo lớn cỡ nào ở rìa ảnh

Để thấy bậc độ lớn, lấy một ống kính góc rộng phổ thông. Giả sử $f = 600$ px, ảnh 1280×960 , và $k_1 = -0,30$ (giá trị điển hình cho góc rộng; *không phải số đo của một ống kính cụ thể*). Xét một điểm ở rìa ngang, cách tâm $r_{\text{px}} = 400$ px.

Toạ độ chuẩn hoá: $r_n = 400/600 = 0,667$, nên $r_n^2 = 0,444$.

Hệ số méo xuyên tâm: $1 + k_1 r_n^2 = 1 - 0,30 \times 0,444 = 1 - 0,133 = 0,867$.

Điểm bị kéo vào trong: bán kính méo là $0,667 \times 0,867 = 0,578$ trong toạ độ chuẩn hoá, tức 347 px.
Độ lệch: $400 - 347 = 53$ px.

Năm mươi ba pixel. Đặt cạnh nhiều định vị góc $\sigma = 0,2$ px, méo lớn hơn nhiều hơn 250 lần. Đây là lý do khử méo không phải một bước tinh chỉnh mà là một bước bắt buộc: bỏ qua nó thì mọi thứ khác trong cây là vô nghĩa.

Con số quan trọng hơn cho mục đích của ta là *méo còn sót lại sau khi khử*. Nếu mô hình đúng dạng và hiệu chuẩn tốt, phần sót cỡ 0,1–0,3 px — cùng bậc với σ , chấp nhận được. Nếu mô hình *sai dạng* — hai hệ số cho ống kính góc rộng — phần sót có thể vẫn là vài pixel ở rìa, và vì nó có cấu trúc chứ không ngẫu nhiên nên nó thành một thiên lệch phụ thuộc vị trí. Mục 6 bàn cách phát hiện.

CẠM BẮY

Có một lỗi cài đặt cực kỳ phổ biến khi tự viết code khử méo, và nó gây ra triệu chứng rất khó truy: *áp công thức méo lên toạ độ pixel thay vì lên toạ độ chuẩn hoá*.

Công thức (1.1) định nghĩa trên (x_n, y_n) , tức sau khi chia cho Z nhưng *trước* khi nhân K . Nếu bạn tính $r^2 = u^2 + v^2$ bằng pixel thay vì $r^2 = x_n^2 + y_n^2$, kết quả sai theo hệ số f^2 — tức sai hàng trăm nghìn lần ở số hạng $k_1 r^2$.

Triệu chứng: hiệu chuẩn vẫn “hội tụ” vì bộ tối ưu sẽ tìm ra một k_1 nhỏ bù lại tỉ lệ, và residual trông không tệ. Nhưng các hệ số ra sẽ khác vài bậc độ lớn so với giá trị điển hình, và mô hình sẽ ngoại suy sai kinh khủng ở những vùng ảnh không có dữ liệu hiệu chuẩn.

Phép kiểm rẻ: hệ số k_1 của ống kính thường nằm trong khoảng -1 tới $+1$. Nếu bạn thấy $k_1 \sim 10^{-7}$ thì gần như chắc chắn bạn đang tính r bằng pixel.

1.5 Điểm chính sai: nguồn thiên lệch xoay không có triệu chứng

Mục này bàn một cơ chế mà tôi cho là bị đánh giá thấp nhất trong toàn bộ chuỗi đo, và nó xứng đáng được nhìn kỹ.

Điểm chính (c_x, c_y) là giao của trục quang với mặt phẳng ảnh. Trục giác nói nó ở giữa ảnh, và hầu hết người ta khởi tạo nó bằng $(N_x/2, N_y/2)$. Thực tế nó lệch: sai lệch lắp ráp cảm biến so với ống kính cỡ vài chục micromet là bình thường, và với pixel 3 μm thì vài chục micromet là *cả chục pixel*.

Vì sao sai lệch ấy nguy hiểm? Hãy xem nó làm gì với phép chiếu. Giả sử giá trị thật là c_x nhưng ta dùng $\hat{c}_x = c_x + \Delta$. Toạ độ chuẩn hoá mà ta tính ra từ một pixel đo được u là

$$\hat{x}_n = \frac{u - \hat{c}_x}{f} = \frac{u - c_x}{f} - \frac{\Delta}{f} = x_n - \frac{\Delta}{f}.$$

Nghĩa là: *mọi* tia nhìn đều bị dịch đi cùng một lượng góc Δ/f radian. Đây không phải nhiều; đây là một phép quay hệ thống của toàn bộ chùm tia.

Với $\Delta = 10$ px và $f = 600$ px: $\Delta/f = 0,0167$ rad = $0,95^\circ$. Gần một độ — và ngân sách góc của ta là $0,5^\circ$.

Bây giờ đến phần tinh tế, và nó là phần giải thích vì sao triệu chứng khó truy. Nếu sai lệch ấy *chỉ* là một phép quay hệ thống của cả camera, thì nó vô hại trong bài toán docking theo teach-by-showing: nó xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu ($A/05$ mục 8). Nhưng nó không chỉ là một phép quay, vì x_n vào công thức chiếu *cùng* với phép chia cho Z . Hậu quả là hiệu

ứng của Δ lên pose ước lượng phụ thuộc vào *tag đang nằm ở đâu trong khung hình và ở khoảng cách nào* — một tag ở giữa ảnh chịu ảnh hưởng khác một tag ở rìa.

Triệu chứng thực tế theo đó: pose sai một lượng thay đổi trơn theo tư thế, không nhảy, không nhiễu, và không có gì trong ảnh trông bất thường. Nó trông hết như một sai số hand-eye, hoặc như một constellation sai — và đó là lý do người ta hay đi tìm nhầm chỗ.

Ghi chú trung thực về mức kiểm chứng. Tôi dẫn cơ chế này bằng cách viết lại phép chiếu (cửa a) và tôi tin nó đúng về cấu trúc. Tôi *chưa* chạy mô phỏng để đo biên độ thiên lệch pose theo Δ (cửa b), và tôi *chưa* tìm được một nguồn phát biểu tường minh đúng dạng này để đối chiếu (cửa c). Vì vậy phần định lượng của mục này nên được coi là ước lượng bậc độ lớn của tôi chứ không phải kết quả đã kiểm. Đây là mục đứng đầu danh sách nợ kiểm chứng của chương trong 99-chua-biet.md.

CẦN NHỚ

Trong bốn tham số nội, f_x và f_y được ước lượng rất tốt bởi mọi quy trình hiệu chuẩn, còn c_x và c_y thì *không*: chúng tương quan mạnh với tư thế của target, và nếu dữ liệu hiệu chuẩn chỉ gồm các ảnh chụp gần vuông góc thì bộ tối ưu không tách được “điểm chính lệch sang trái” khỏi “target nghiêng sang phải”.

Hệ quả thực hành trực tiếp, và nó sẽ được nhắc lại ở B/12: dữ liệu hiệu chuẩn *phải* gồm các ảnh ở tư thế nghiêng mạnh và target phải phủ tới sát bốn góc khung hình. Một bộ ảnh hiệu chuẩn “đẹp” — target luôn ở giữa, luôn vuông góc — cho residual nhỏ và điểm chính sai.

Đây là một trường hợp cụ thể của nguyên tắc chung: residual nhỏ không chứng minh tham số đúng; nó chỉ chứng minh mô hình khớp được dữ liệu bạn đã cho.

1.6 Sai số mô hình so với sai số tham số

Hai loại sai lệch rất khác nhau thường bị gộp làm một, và phân biệt chúng là kỹ năng chẩn đoán quan trọng nhất của chương.

Sai số tham số (*parameter error*) là khi dạng hàm đúng nhưng hệ số ước lượng chưa chính xác — chẳng hạn mô hình Brown–Conrady đúng cho ống kính của bạn nhưng k_1 lệch 2% vì dữ liệu hiệu chuẩn ít. Loại này *chữa được bằng thêm dữ liệu*: chụp thêm ảnh, phủ rộng hơn, và hệ số hội tụ về giá trị đúng.

Sai số mô hình (*model error*) là khi dạng hàm bản thân nó không đủ để mô tả ống kính — chẳng hạn dùng hai hệ số radial cho một ống kính 120°. Loại này *không* chữa được bằng thêm dữ liệu. Bộ tối ưu sẽ tìm ra bộ hệ số tốt nhất trong họ hàm bạn đã chọn, và nếu họ hàm ấy không chứa hàm đúng thì “tốt nhất” vẫn sai. Thêm một nghìn ảnh cho ra cùng một kết quả sai, chỉ với khoảng tin cậy hẹp hơn — một tình huống đặc biệt nguy hiểm vì nó làm bạn tin hơn vào một con số sai.

Phép thử phân biệt hai loại rất đơn giản và nó là kỹ thuật chẩn đoán giá trị nhất trong chương: *vẽ sai số tái chiếu theo vị trí trên ảnh*.

Nếu chỉ có sai số tham số, các vectơ residual trông như nhiễu trắng — hướng ngẫu nhiên, độ lớn đều, không có cấu trúc không gian. Nếu có sai số mô hình, chúng tạo thành một *mẫu hình*: thường là các vectơ hướng xuyên tâm với độ lớn tăng dần theo bán kính, hoặc các xoáy có tổ chức ở bốn góc ảnh. Kỹ thuật này đã được dùng trong hiệu chuẩn camera chính xác từ lâu [6] và nó rõ đến mức không có lý do gì để bỏ qua.

CẠM BÃY

Sai số tái chiếu trung bình (RMS reprojection error) là con số mà mọi công cụ hiệu chuẩn in ra, và nó là con số *tệ nhất* để đánh giá chất lượng hiệu chuẩn khi dùng một mình. Ba lý do.

Thứ nhất, nó là một trung bình trên toàn ảnh, nên một vùng rìa sai 2 px chìm mất trong hàng nghìn điểm ở giữa sai 0,1 px.

Thứ hai, nó *giảm* khi bạn thêm tham số, kể cả tham số vô nghĩa — đó là bản chất của khớp bình phương tối thiểu. Một mô hình hữu tỉ 8 tham số luôn cho RMS nhỏ hơn Brown–Conrady 5 tham số trên cùng dữ liệu, và điều đó không nói gì về việc mô hình nào đúng hơn.

Thứ ba, và nghiêm trọng nhất: nó đo độ khớp trên *dữ liệu hiệu chuẩn*, không đo độ đúng khi ngoại suy sang vùng không có dữ liệu. Nếu target của bạn không bao giờ chạm tới góc khung hình thì RMS không biết gì về góc khung hình, và tag của robot lúc tiếp cận thì rất có thể nằm đúng ở đó.

Thay vào đó hãy nhìn: bản đồ residual theo vị trí (có cấu trúc hay không), độ phủ của dữ liệu hiệu chuẩn (có chạm bốn góc không), và độ ổn định của tham số qua nhiều lần hiệu chuẩn độc lập.

1.7 Khử méo: toàn ảnh hay từng điểm

Hai cách, và chúng không tương đương — đây là câu trả lời cho Q4.

Cách một, khử méo toàn ảnh rồi chạy detector. Tạo một ảnh mới trong đó mọi pixel đã được đặt lại đúng vị trí pinhole, rồi chạy detector trên ảnh ấy. Ưu điểm: mọi giả định của detector về đường thẳng trở thành đúng, nên khớp cạnh hợp lệ. Nhược điểm: phép khử méo đòi *nội suy* giá trị pixel ở các vị trí không nguyên, và nội suy làm nhoè ảnh — tức làm tăng σ . Với nội suy song tuyến tính (bilinear), mức nhoè đáng kể; với nội suy bậc cao hơn thì đỡ hơn nhưng chậm hơn.

Cách hai, chạy detector trên ảnh gốc rồi khử méo cho bốn toạ độ góc. Ưu điểm: không nội suy, nên không mất độ sắc nét, nên σ giữ nguyên; và rẻ hơn nhiều vì chỉ xử lý bốn điểm thay vì một triệu. Nhược điểm: detector đang khớp đường thẳng lên cạnh cong, nên bốn góc nó trả về đã sai *trước khi* bạn khử méo — và khử méo cho một điểm sai chỉ cho ra một điểm sai ở chỗ khác.

Vậy cách nào đúng? Câu trả lời phụ thuộc vào độ lớn của méo trong vùng tag chiếm, và nó dẫn tới một quy tắc thực hành khá rõ. Nếu tag nhỏ và nằm ở vùng ảnh có méo gần tuyến tính cục bộ — tức gần tâm, hoặc ống kính ít méo — thì cách hai tốt hơn, vì phần cong trên chiều dài một cạnh tag là không đáng kể còn lợi ích về độ sắc nét thì thật. Nếu tag lớn hoặc nằm ở rìa ảnh của một ống kính góc rộng, cách một bắt buộc.

Có một cách thứ ba, tốt hơn cả hai nhưng ít được dùng: *đưa mô hình méo vào ngay trong hàm sai số tái chiếu* của bước tinh chỉnh PnP. Tức là không khử méo gì cả; thay vào đó, khi dự đoán vị trí của một góc, ta chiếu điểm 3D qua pinhole rồi *áp méo*, và so với toạ độ đo được trên ảnh gốc. Cách này đúng về mặt mô hình và nó không mất độ sắc nét; giá phải trả là bước phát hiện ban đầu vẫn dùng ảnh gốc nên vẫn có vấn đề khớp thẳng-lên-cong ở tầng detector. Nó là cách mà các hộp công cụ hiệu chuẩn chất lượng cao dùng, và nó là lựa chọn đúng khi bạn kiểm soát được toàn bộ chuỗi.

1.8 Giới hạn và chế độ hồng

Giả thiết tâm chiếu là một điểm. Ống kính thật có *đồng tử vào* (entrance pupil) với kích thước hữu hạn, và vị trí hiệu dụng của nó dịch chuyển theo góc tới. Với chụp gần — và docking là chụp gần — hiệu ứng này không hoàn toàn bỏ qua được. Nó thường bị hấp thụ vào các hệ số méo trong quá trình hiệu chuẩn, nghĩa là các hệ số ấy trở thành phụ thuộc khoảng cách. Hệ quả thực hành: *hiệu chuẩn ở khoảng cách gần với khoảng cách vận hành*.

Giả thiết mô hình méo không đổi. Ống kính trôi theo nhiệt độ: giãn nở nhiệt làm tiêu cự đổi và làm điểm chính dịch. Với ống kính nhựa và chênh lệch nhiệt độ vài chục độ, hiệu ứng có thể ở mức pixel. B/12 bàn chuyện này; ở đây chỉ cần ghi nhận rằng “đã hiệu chuẩn” không phải một trạng thái vĩnh viễn.

Giả thiết lấy nét cố định. Đổi tiêu điểm làm đổi cả f lẫn hệ số méo. Với ống kính lấy nét tự động, mọi tham số nội của bạn chỉ đúng ở đúng vị trí lấy nét lúc hiệu chuẩn. Đây là lý do hệ thống đo lường dùng ống kính khoá nét, và là lý do nên dán keo vào vòng lấy nét sau khi hiệu chuẩn.

Giả thiết cảm biến phẳng và lưới pixel đều. Đúng ở mức rất tốt với cảm biến hiện đại, nên đây hiếm khi là vấn đề — ghi ra để hoàn chỉnh danh sách giả thiết chứ không phải để lo lắng.

1.9 Thực hành

Bốn việc, theo thứ tự.

Một, ước lượng f từ FOV ngay khi thiết kế, bằng công thức ở mục 2. Bạn cần nó cho ngân sách sai số ở C/15 và bạn cần nó *trước* khi mua camera.

Hai, chọn mô hình méo theo FOV chứ không theo mặc định của thư viện, dùng bảng 1.1. Dùng mô hình quá nghèo là sai số mô hình và nó không chứa được bằng dữ liệu; dùng mô hình quá giàu là rủi ro khớp quá mức và nó chứa được bằng dữ liệu phủ rộng.

Ba, sau mỗi lần hiệu chuẩn, vẽ bản đồ residual theo vị trí ảnh trước khi nhìn con số RMS. Nếu thấy cấu trúc xuyên tâm hoặc xoáy ở góc thì mô hình sai dạng, và thêm ảnh không cứu được.

Bốn, kiểm c_x, c_y bằng cách hiệu chuẩn lại vài lần với các bộ ảnh khác nhau và xem chúng có ổn định không. Nếu điểm chính nhảy hàng chục pixel giữa các lần thì dữ liệu của bạn không đủ đa dạng về tư thế, và con số bạn đang dùng là ngẫu nhiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
● ● ● Ba bước của mô hình chiếu	Thế giới sang camera (R, t) ; chia cho Z ra toạ độ chuẩn hoá (bước phi tuyến duy nhất); áp méo rồi nhân K ra pixel. Méo định nghĩa trên toạ độ <i>chuẩn hoá</i> , không phải pixel.
● ● ● Tiêu cự tính bằng pixel	$f_x = F/s_x$, một tỉ số <i>không thứ nguyên</i> . Ước lượng nhanh từ FOV: $f_x \approx (N_x/2)/\tan(\theta_{\text{FOV}}/2)$. Ví dụ 1280 px, $60^\circ \Rightarrow 1109$ px.
● ● ● Vì sao méo phá detector marker đặc biệt	Detector khớp <i>đường thẳng</i> lên cạnh tag rồi lấy giao [3], nhưng méo xuyên tâm biến đường thẳng thành đường cong. Khớp thẳng lên cong cho đường sai, và giao hai đường sai bị khuếch đại.
● ● ● Méo là nhiều hay thiên lệch?	<i>Thiên lệch</i> . Với tag ở một vị trí cố định, méo đẩy cạnh theo cùng hướng ở mọi khung. Không chia cho $\sqrt{N}$ — đây là nguồn “hệ thống” ở A/05 mục 8.
● ● ● Độ lớn méo ở rìa	Ví dụ: $f = 600$, $k_1 = -0,3$, điểm cách tâm 400 px $\Rightarrow$ lệch 53 px. Lớn hơn $\sigma = 0,2$ px hơn 250 lần. Nên khử méo là bắt buộc, không phải tinh chỉnh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Điểm chính sai gây gì	Dịch mọi tia nhìn đi Δ/f radian. Với $\Delta = 10$ px, $f = 600$: $0,95^\circ$ — gần gấp đôi ngân sách góc. Không có triệu chứng thị giác nào.
●●● Vì sao c_x, c_y khó hiệu chuẩn	Chúng tương quan mạnh với tư thế target. Ảnh chỉ chụp gần vuông góc thì không tách được “điểm chính lệch trái” khỏi “target nghiêng phải”. Phải có tư thế nghiêng mạnh và phủ tới bốn góc.
●●● Sai số mô hình so với sai số tham số	Tham số: dạng hàm đúng, hệ số lệch — chứa được bằng thêm dữ liệu. Mô hình: dạng hàm không đủ — <i>không</i> chứa được bằng dữ liệu, thêm nghìn ảnh vẫn sai với khoảng tin cậy hẹp hơn.
●●● Phép thử phân biệt hai loại	Vẽ residual theo vị trí ảnh [6]. Nhiều trắng $\Rightarrow$ sai số tham số. Mẫu hình xuyên tâm hoặc xoáy ở góc $\Rightarrow$ sai số mô hình.
●● Vì sao RMS reprojection là chỉ số tệ	Trung bình che vùng rìa; nó giảm khi thêm tham số bất kể tham số có nghĩa hay không; và nó đo độ khớp trên dữ liệu hiệu chuẩn chứ không đo độ đúng khi ngoại suy sang vùng chưa có dữ liệu.
●● Khử méo toàn ảnh hay từng điểm	Toàn ảnh: giả định detector thành đúng, nhưng nội suy làm nhoè $\Rightarrow$ tăng σ . Từng điểm: giữ độ sắc nét, nhưng góc đã sai từ khâu khớp cạnh. Tốt hơn cả: đưa méo vào hàm sai số tái chiếu của PnP.
●● Chọn mô hình méo theo FOV	Dưới 50° : 2 hệ số radial. $50\text{--}90^\circ$: Brown–Conrady 5 tham số. $90\text{--}120^\circ$: hữu tỉ. Trên 120° : KB/fisheye [4]. Dùng nhằm theo hướng nghèo là sai số mô hình.
● Lỗi cài đặt phổ biến nhất	Tính r^2 bằng pixel thay vì tọa độ chuẩn hoá — sai theo hệ số f^2 . Triệu chứng: hiệu chuẩn vẫn hội tụ nhưng $k_1 \sim 10^{-7}$ thay vì cỡ -1 tới $+1$.
● Ba giả thiết hay bị quên	Tâm chiếu là điểm (sai khi chụp gần — nên hiệu chuẩn ở khoảng cách vận hành); mô hình không đổi (sai vì trôi nhiệt); lấy nét cố định (đổi nét là đổi cả f lẫn hệ số méo — nên khoá nét).

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>calibrateCamera</code> , <code>undistortPoints</code>	Cài Brown–Conrady và hữu tỉ; <code>undistortPoints</code> chính là “cách hai” ở mục 7	Đọc kỹ thứ tự tham số <code>distCoeffs</code> : k_1, k_2, p_1, p_2, k_3 — không phải thứ tự trong công thức
Kalibr [7]	Hỗ trợ nhiều mô hình méo và báo cáo residual theo vị trí ảnh	Công cụ duy nhất trong danh sách mặc định cho bạn bản đồ residual ở mục 6
deltille [8]	Target lưới tam giác cho định vị góc chính xác hơn, dùng khi cần intrinsic chất lượng cao	Nổi sang B/08; target tốt hơn cho cùng thuật toán
Hiệu chuẩn từ target phẳng [2]	Phương pháp nền của mọi hộp công cụ; giải thích vì sao cần nhiều tư thế nghiêng	Đọc §3 để thấy vì sao tư thế fronto-parallel không cho thông tin về c_x, c_y
Double Sphere [5]	Mô hình ít tham số cho FOV rất rộng	Lựa chọn hiện đại thay cho KB khi cần ít tham số

MINI PROJECT · Đo méo còn sót lại sau khi khử, và vẽ bản đồ residual · 1–2 ngày

Mục tiêu	biến câu “hiệu chuẩn của tôi có tốt không” thành một bản đồ hai chiều, và tự tay nhìn thấy sự khác nhau giữa sai số mô hình và sai số tham số.
Dữ liệu	một camera thật và một target ChArUco hoặc chessboard in trên nền cứng phẳng. Nếu chưa có phần cứng, mô phỏng: sinh ảnh bằng một mô hình méo “thật” (chẳng hạn hữu tỉ 8 tham số) rồi hiệu chuẩn bằng một mô hình nghèo hơn (2 hệ số) — cách này còn rõ hơn vì bạn biết chân trị.
Các bước	chụp ít nhất 30 ảnh phủ kín khung hình, <i>bắt buộc</i> có ảnh mà target chạm sát bốn góc và có ảnh nghiêng trên 45° ; hiệu chuẩn với ba mô hình khác nhau (2 hệ số, Brown–Conrady đầy đủ, hữu tỉ); với mỗi mô hình, lấy vectơ residual của từng điểm và vẽ chúng như một trường vectơ trên mặt phẳng ảnh, phóng đại 50 lần; so sánh cả ba bản đồ; đồng thời ghi lại c_x, c_y của ba lần và của các tập con ảnh khác nhau.
Xong khi	bạn có ba bản đồ residual đặt cạnh nhau và phát biểu được một câu kiểu “với mô hình 2 hệ số, residual có cấu trúc xuyên tâm rõ với độ lớn tới X px ở bán kính $r > Y$; với Brown–Conrady đầy đủ thì cấu trúc biến mất”. Kiểm tra chéo bắt buộc: nếu c_x, c_y nhảy hơn 5 px giữa các tập con ảnh, thì dữ liệu của bạn chưa đủ đa dạng tư thế và mọi kết luận về mô hình méo đều chưa đáng tin — chụp thêm ảnh nghiêng trước đã.
Nối với	B/12 cho quy trình hiệu chuẩn đầy đủ; A/05 dùng f mà bạn vừa đo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì RMS là một trung bình trên toàn ảnh và trên toàn bộ dữ liệu hiệu chuẩn, nên nó có ba điểm mù. Nó che vùng rìa: hàng nghìn điểm giữa ảnh sai 0,1 px chìm chết vài chục điểm rìa sai 2 px. Nó luôn giảm khi thêm tham số, kể cả tham số vô nghĩa — nên so RMS giữa hai mô hình có số tham số khác nhau là vô nghĩa. Và quan trọng nhất, nó đo độ khớp *trên dữ liệu bạn đã cho* chứ không đo độ đúng ở vùng không có dữ liệu, mà tag của robot lúc tiếp cận rất có thể nằm đúng ở vùng ấy. Chỗ cần nhìn là bản đồ residual theo vị trí ảnh: nhiều trắng thì ổn, còn thấy cấu trúc xuyên tâm hay xoáy ở bốn góc thì mô hình sai dạng và thêm ảnh không cứu được.
- A2. Không phải 3 px, và thường là lớn hơn. Lý do là detector không lấy trực tiếp vị trí góc mà khớp một đường thẳng lên mỗi cạnh rồi lấy *giao* của hai đường. Méo làm cạnh cong, nên đường thẳng khớp lên nó bị lệch cả về vị trí lẫn về *hướng*; và một sai lệch hướng nhỏ của hai đường giao nhau sẽ chuyển thành sai lệch vị trí lớn ở giao điểm, với hệ số khuếch đại phụ thuộc góc giao — càng chệch thì càng khuếch đại. Cần thêm một điều: sai lệch này là thiên lệch chứ không phải nhiễu, nên lấy trung bình nghìn khung không giúp gì.
- A3. Nó dịch *mọi* tia nhìn đi một góc Δ/f radian — với $\Delta = 10$ px và $f = 600$ px thì là $0,95^\circ$, tức gần gấp đôi ngân sách góc của cây. Hậu quả thay đổi theo vị trí tag trong khung hình vì x_n vào phép chiếu cùng với phép chia cho Z , nên hiệu ứng của Δ lên sáu tham số pose không phải một phép quay thuần mà phụ thuộc cả vị trí lẫn khoảng cách. Điều làm nó nguy hiểm đặc biệt: nó không có triệu chứng thị giác nào — không ai nhìn ảnh mà phát hiện được điểm chính sai — và triệu chứng ở đầu ra thì trông hệt như sai số hand-eye hoặc constellation sai, nên người ta hay đi tìm nhầm chỗ.
- A4. Khử méo toàn ảnh chính xác hơn về mặt mô hình, vì nó làm giả định “cạnh là đường thẳng” của detector trở thành đúng; nhưng nó đòi nội suy pixel, mà nội suy làm nhoè, mà nhoè làm tăng σ . Khử méo từng điểm nhanh hơn nhiều và giữ nguyên độ sắc nét, nhưng nó không sửa được việc bốn góc đã bị định vị sai từ khâu khớp cạnh trên ảnh cong. Hai câu trả lời khác nhau vì hai câu hỏi đo hai thứ khác nhau: một câu hỏi về tính đúng của mô hình, một câu hỏi về chi phí tính toán và về σ . Quy tắc thực hành: tag nhỏ và gần tâm thì dùng cách hai; tag lớn hoặc ở rìa ống kính góc rộng thì bắt buộc cách một; và tốt hơn cả hai là đưa mô hình méo thẳng vào hàm sai số tái chiếu của bước tinh chỉnh PnP.
- A5. Vì residual nhỏ chỉ chứng minh mô hình khớp được *dữ liệu bạn đã cho*, không chứng minh nó mô tả đúng ống kính. Với ống kính 120° , phép chiếu pinhole $r_n = \tan \theta$ đã gần phân kỳ, và không đa thức bậc bốn nào theo dõi kịp — nhưng bộ tối ưu vẫn tìm ra bộ (k_1, k_2) tốt nhất trong họ hàm nghèo ấy và báo về một con số RMS nghe được. Đây là sai số mô hình, và thêm một nghìn ảnh chỉ làm khoảng tin cậy của các hệ số hẹp lại quanh một giá trị vẫn sai — tệ hơn, vì bạn sẽ tin nó hơn. Dấu hiệu lộ ra: bản đồ residual có cấu trúc xuyên tâm với độ lớn tăng theo

bán kính; và nếu bạn hiệu chuẩn hai lần với hai tập ảnh phủ vùng khác nhau, các hệ số sẽ khác nhau đáng kể vì mỗi lần mô hình đang “cố” khớp một vùng khác.

NÓI LẠI THẬT ĐƠN GIẢN

Một cái camera về bản chất là một cái hộp có một lỗ nhỏ. Ánh sáng từ mọi vật chui qua cái lỗ ấy rồi đập vào thành sau của hộp, và ảnh hiện ra. Nếu cái lỗ thật sự nhỏ như một điểm và ánh sáng đi thẳng, thì mọi thứ rất dễ tính: cứ kẻ một đường từ vật qua cái lỗ, nó đập vào đâu thì ảnh ở đó. Vấn đề là camera thật không có lỗ nhỏ, nó có thấu kính — vì lỗ nhỏ cho quá ít ánh sáng. Thấu kính thu được nhiều sáng hơn nhưng nó bẻ cong tia sáng, và nó bẻ tia ở rìa nhiều hơn tia ở giữa. Kết quả là một cái cửa sổ hình chữ nhật chụp bằng ống kính góc rộng trông phình ra như quả trứng.

Với phần lớn bài toán thị giác thì chuyện đó không quá tệ, vì người ta chỉ cần biết từng điểm nằm đâu, và có công thức để trả điểm về đúng chỗ. Nhưng cái tag thì khác, và đây là chỗ cần chú ý. Máy tính tìm góc của tag bằng cách nhìn bốn cạnh, kẻ bốn đường thẳng dọc theo chúng, rồi lấy chỗ các đường cắt nhau. Cách ấy rất khôn vì nó dùng cả trăm pixel dọc cạnh chứ không chỉ vài pixel ở góc. Nhưng nó chỉ đúng nếu cạnh thật sự thẳng — mà ống kính vừa làm chúng cong.

Kẻ một đường thẳng dọc theo một đường cong thì được một đường lệch. Lấy chỗ hai đường lệch cắt nhau thì được một điểm lệch nhiều hơn nữa — giống như hai cây gậy dài chạm nhau ở đầu: nghiêng mỗi cây một chút thôi, đầu gậy đã trượt đi khá xa.

Còn một thứ nữa, kín đáo hơn và vì thế phiền hơn. Cái lỗ của camera đáng lẽ nằm đúng giữa thành sau, nhưng khi lắp ráp thì nó lệch đi một chút. Chuyện lệch ấy không làm ảnh trông khác đi chút nào — bạn không có cách nào nhìn một tấm ảnh rồi bảo “lỗ này lệch sang trái mười pixel”, vì trong ảnh không có gì để so. Nhưng máy tính lại dùng vị trí của cái lỗ để tính hướng nhìn, nên lệch mười pixel tương đương với tính sai hướng gần một độ. Và một độ thì đã quá ngân sách của chúng ta rồi.

Bài học chung của cả chương: có những sai lệch làm ảnh trông xấu, và có những sai lệch không làm ảnh trông khác gì cả. Loại thứ hai mới là loại phải cẩn thận, vì không có gì nhắc bạn rằng chúng đang ở đó.

Chỗ không rút gọn được: việc chọn mô hình méo nào cho ống kính nào thì không có quy tắc đơn giản đúng cho mọi trường hợp. Bảng trong chương là hướng dẫn theo góc nhìn, và nó đúng phần lớn thời gian, nhưng cách chắc chắn duy nhất là hiệu chuẩn với vài mô hình rồi nhìn bản đồ residual xem cái nào hết cấu trúc.

Một câu: Ống kính bẻ cong đường thẳng, còn máy tính thì cứ tưởng chúng thẳng — và cái lỗ lệch mười pixel thì không ai nhìn thấy nhưng nó ăn gần một độ.

1.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *méo chưa khử làm cong cạnh tag, và việc khớp đường thẳng lên cạnh cong khuếch đại sai số tại giao điểm* — bị bác bỏ nếu một mô phỏng cho thấy sai số góc do méo gây ra nhỏ hơn hoặc bằng độ lớn của méo tại vị trí góc đó. Tôi dẫn mệnh đề bằng hình học (cửa a) và bằng ví dụ số cho độ lớn méo (cửa b cho phần độ lớn), nhưng phần *khuếch đại tại giao điểm* thì tôi chưa đo bằng mô phỏng — đó là một nợ kiểm chứng.

Mệnh đề thứ hai — *điểm chính sai Δ px gây thiên lệch hướng cỡ Δ/f radian* — là mệnh đề yếu nhất về mặt kiểm chứng trong chương, như đã ghi thẳng ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy sai số pose do Δ gây ra được hấp thụ gần hết vào các tham số khác và biên độ còn lại nhỏ hơn nhiều so với Δ/f . Đây là mệnh đề tôi khuyên kiểm sớm nhất, vì nếu nó sai thì mức ưu tiên tôi gán cho việc hiệu chuẩn điểm chính ở B/12 là quá cao.

Mệnh đề thứ ba — *sai số mô hình không chữa được bằng thêm dữ liệu* — là mệnh đề chắc chắn nhất, vì nó là một mệnh đề về bản chất của khớp trong một họ hàm hạn chế, không phải một mệnh

để thực nghiệm. Nó “bị bác bỏ” chỉ nếu ai đó chỉ ra rằng họ hàm bạn dùng thật ra có chứa hàm đúng — tức là chỉ ra rằng bạn chẩn đoán nhầm, chứ không phải mệnh đề sai.

Cuối cùng, một mệnh đề mềm: chương này khẳng định rằng *trong thực tế, điểm chính là tham số nội gây hại nhất*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã hiệu chuẩn điểm chính cẩn thận, sai số xoay không cải thiện đáng kể so với khi dùng giá trị mặc định giữa ảnh. (Tôi phát biểu dựa trên cấu trúc của phép chiếu và trên sự thiếu triệu chứng của lỗi này, không dựa trên thống kê ca hòng thực tế.)

1.11 Kết nối

Chương này là gốc của Phần A và nó cấp vật liệu cho gần như mọi chương sau.

Nối xuống dưới trong Phần A. A/02-homography-va-hình-hoc-phang dùng ma trận K trực tiếp trong phân rã $H = K[r_1 \ r_2 \ t]$, và nó giả định ảnh đã được khử méo — một giả định mà chương này vừa cho thấy không hiển nhiên. A/03-pnp-va-uoc-luong-tu-the dùng mô hình chiếu ở mục 2 làm hàm $\hat{u}(\xi)$ trong sai số tái chiếu, và mục 7 ở đây bàn chỗ đưa méo vào hàm ấy. A/05-lan-truyen-sai-so-pixel-sang-pose lấy f làm mẫu số của công thức sai số ngang, và nó lấy từ mục 6 ở đây luận điểm quan trọng rằng “mô hình đúng” là một giả thiết chứ không phải một sự thật — khi giả thiết ấy vi phạm, hiệp phương sai mà chương 5 tính ra là sai theo chiều nguy hiểm.

Nối sang Phần B. B/07-detection-pipeline là chương mô tả chi tiết cơ chế khớp cạnh mà mục 4 ở đây dựa vào; đọc hai mục ấy cùng nhau thì thấy rõ vì sao méo và detector tương tác xấu. B/08-corner-refinement quyết định σ , và mục 7 ở đây cảnh báo rằng khử méo toàn ảnh làm tăng chính σ ấy — một đánh đổi phải cân. B/12-calibration-cho-docking là chương nhận trực tiếp và đầy đủ nhất: toàn bộ mục 5 và mục 6 ở đây trở thành quy trình cụ thể ở đó, gồm cả yêu cầu về độ phủ dữ liệu và phép thử residual có cấu trúc.

Nối sang Phần C. C/15-ngan-sach-sai-so có một dòng riêng cho “méo còn sót lại” và một dòng riêng cho “sai số intrinsic”, và cả hai dòng ấy được định giá bằng các cơ chế ở chương này. Đáng chú ý là chúng thuộc loại *hệ thống* theo phân loại ở A/05 mục 8, nên chúng không giảm khi lấy trung bình nhiều khung — và đó là lý do chúng thường chiếm phần lớn ngân sách trong một hệ thống đã tối ưu tốt phần nhiều.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao phép méo xuyên tâm biến một đường thẳng thành đường cong, nhưng có đúng một họ đường thẳng vẫn thẳng sau khi méo — họ nào, và vì sao?
2. Detector khớp đường thẳng lên cạnh tag. Nếu chính xác chuỗi ba bước biến “ảnh còn méo 3 px” thành “pose sai”, và chỉ ra bước nào khuếch đại sai số.
3. Bạn hiệu chuẩn hai lần với hai tập ảnh khác nhau và được hai giá trị c_x cách nhau 15 px. Điều đó nói gì về dữ liệu của bạn, và vì sao f_x của hai lần lại có thể gần như trùng nhau?
4. Vì sao thêm tham số vào mô hình méo luôn làm RMS reprojection giảm, và vì sao điều đó không chứng minh mô hình tốt hơn?
5. Một đồng nghiệp khử méo toàn ảnh bằng nội suy song tuyến tính rồi thấy σ tăng. Giải thích cơ chế, và nêu hai cách giữ được cả tính đúng của mô hình lẫn độ sắc nét.
6. Với $\Delta c_x = 10$ px và $f = 600$ px, thiên lệch hướng là bao nhiêu độ? Vì sao thiên lệch ấy phần lớn triệt tiêu trong sơ đồ teach-by-showing nhưng *không* triệt tiêu hoàn toàn?
7. Nêu ba giả thiết ngầm của mô hình pinhole cộng méo mà chương này liệt kê, và với mỗi giả thiết, một tình huống docking cụ thể làm nó bị vi phạm.

Nguồn

- [1] D. C. Brown, "Close-Range Camera Calibration," *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, "AprilTag 2: Efficient and Robust Fiducial Detection," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, "A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, "The Double Sphere Camera Model," in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, "Accurate Calibration of CCD-Cameras," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, "Unified Temporal and Spatial Calibration for Multi-Sensor Systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., "Deltile Grids for Geometric Camera Calibration," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.